From MLP to CNN: Spatial Feature Learning and Classifier-Head Efficiency on BloodMNIST

Victoria Tran (My-Uyen Tran)
2026 | Independent educational research project

Abstract. This report compares one multilayer perceptron (MLP) and two small convolutional neural networks (CNNs) on the eight-class BloodMNIST dataset. The study uses the official BloodMNIST splits, training-derived RGB normalization, and one fixed training protocol with a single predefined random seed. For each model, the checkpoint with the highest validation macro-F1 was selected and then evaluated once on the held-out test split. The CNN with a Flatten classifier head (M1) achieved the highest test macro-F1 (0.9051), followed by the CNN with Global Average Pooling (M2, 0.8452) and the MLP baseline (M0, 0.8150). Compared with M1, M2 used 55.0% fewer trainable parameters but achieved lower classification performance. These results describe the tested setting only and do not establish a universal ranking of neural network architectures.

1. Introduction and Research Questions

An MLP classifies images after flattening pixels into a one-dimensional vector. A CNN instead learns features from local spatial patterns through convolutional filters. This project uses BloodMNIST as a small educational biomedical image-classification task to compare these two modeling ideas under the same data splits and shared training settings.

The study addresses two research questions:

- **RQ1 — MLP vs. CNN:** How do the tested MLP and CNN architectures compare in classification performance on BloodMNIST under the same official data splits and shared training settings?
- **RQ2 — Flatten vs. GAP:** When the CNN feature-extractor design is held constant, how does replacing a Flatten classifier head with Global Average Pooling (GAP) affect trainable parameter counts and classification performance?

The goal is a small empirical study of model architectures, not a clinically validated diagnostic system.

2. Dataset and Preprocessing

BloodMNIST is part of the MedMNIST v2 benchmark and contains 17,092 RGB blood-cell images of size 28×28 across eight classes [1, 2]. The official train, validation, and test splits were used directly, without additional splitting.

Split	Images	Share	Purpose
Training	11,959	70.0%	Optimize model parameters
Validation	1,712	10.0%	Select the best checkpoint for each model
Test	3,421	20.0%	Final held-out evaluation after checkpoint selection

Images were converted to floating-point tensors with `ToTensor()`. Per-channel RGB means and standard deviations were then computed *from the training split only*: mean (0.794348, 0.659659, 0.696191) and standard deviation (0.215630, 0.241603, 0.117889). These training-derived statistics were used with `Normalize()` for all three splits. No resizing or random data augmentation was applied.

The dataset is class-imbalanced. For example, neutrophils are the largest training class and lymphocytes are the smallest. Macro-F1 is the primary metric because it averages the F1 scores of the eight classes with equal weight; accuracy is reported as a secondary metric.

3. Model Overview

Each model takes an image tensor of shape $(B, 3, 28, 28)$ and produces eight raw logits. M0 is an MLP baseline. M1 and M2 use the same two-block Conv–BatchNorm–ReLU–MaxPool feature-extractor design but different classifier heads.

Model	Architecture summary	Parameters
M0 (MLP)	Flatten → Linear(2352, 256) → Linear(256, 128) → Linear(128, 8); ReLU and Dropout after hidden layers	636,296
M1 (CNN + Flatten)	Two Conv–BatchNorm–ReLU–MaxPool blocks (32 and 64 channels) → Flatten(3,136) → Dropout → Linear(3,136, 8)	44,680
M2 (CNN + GAP)	Same two-block CNN design → GAP(64) → Flatten → Dropout → Linear(64, 8)	20,104

Hidden MLP Linear weights and CNN convolutional weights use He/Kaiming Uniform initialization. Output Linear layers retain PyTorch default initialization. Dropout uses $p = 0.2$. The detailed architectures are shown in Appendix A.

4. Model Implementation and Parameter Counts

The three models were implemented in PyTorch and trained from scratch. M0 flattens each image into 2,352 pixel values and then passes them through two hidden Linear layers. M1 and M2 both generate feature maps of shape $(B, 64, 7, 7)$ after the second MaxPool layer. Their only planned architectural difference is the classifier head.

Model	Classifier input	Final Linear parameters	Total parameters
M0 (MLP)	128	1,032	636,296
M1 (CNN + Flatten)	3,136	25,096	44,680
M2 (CNN + GAP)	64	520	20,104

For an eight-class Linear layer with n input features, the parameter count is $8n + 8$. The output layers of M1 and M2 differ by $25{,}096 - 520 = 24{,}576$ parameters. Because their CNN feature-extractor designs match, this is also the difference in total model parameters (55.0%).

5. Training and Evaluation Protocol

A fixed, single-seed protocol was used. Seed 42 was reset before each independent run. Each model was created from scratch, with a fresh Adam optimizer and a seed-controlled shuffled training DataLoader. All three models used unweighted cross-entropy loss and were trained for all 12 epochs, without early stopping, learning-rate search, or hyperparameter tuning.

Setting	Configuration
Optimizer / learning rate	Adam / 0.001
Batch size / epochs	128 / 12 per model
Random seed	42 (reset for each run)
Loss / augmentation	Cross-entropy / none
Validation metrics	Loss, accuracy, macro-F1 after every epoch
Checkpoint selection	Highest validation macro-F1 (earlier epoch in a tie)
Test metrics	Macro-F1 (primary), accuracy (secondary)

The workflow was as follows: (1) load the official BloodMNIST splits, (2) compute training-only normalization statistics, (3) train each model for 12 epochs, (4) save the best checkpoint based on validation macro-F1, (5) reload the saved checkpoint, and (6) evaluate once on the held-out test set. The test split was not used for checkpoint selection.

6. Experimental Results

Table 1 reports the best validation-selected checkpoints and their held-out test performance.

Table 1: Validation-selected checkpoint and held-out test results (one run per model, seed 42).

Model	Params	Best epoch	Val macro-F1	Test acc.	Test macro-F1
M0	636,296	12	0.8366	0.8381	0.8150
M1	44,680	7	**0.9156**	**0.9152**	**0.9051**
M2	20,104	11	0.8596	0.8702	0.8452

The ranking by both validation and test macro-F1 was **M1** > **M2** > **M0**. M0 had the most parameters but the lowest test macro-F1; M1 achieved the highest score.

7. Training and Validation Behavior

Figures 1–3 show the training/validation cross-entropy losses and validation macro-F1 curves for all three official runs. Each dashed marker identifies the epoch selected using validation macro-F1.

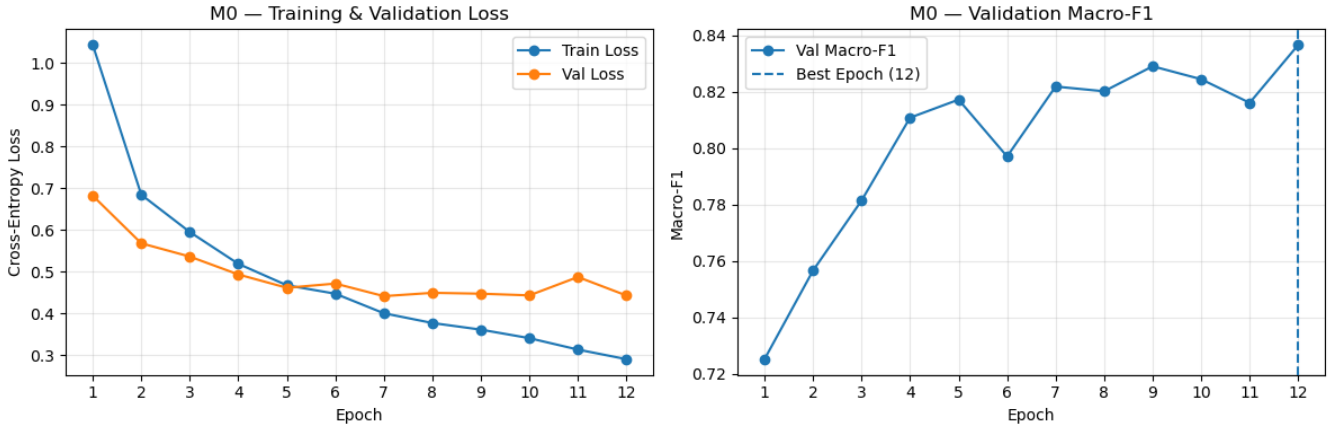


Figure 1: M0 learning curves over 12 epochs. Validation macro-F1 reached its highest value at epoch 12.

M0's training loss decreased across the 12 epochs. Its validation macro-F1 improved overall, with some fluctuations, and reached its maximum at the final epoch. Thus the selected M0 checkpoint was epoch 12.

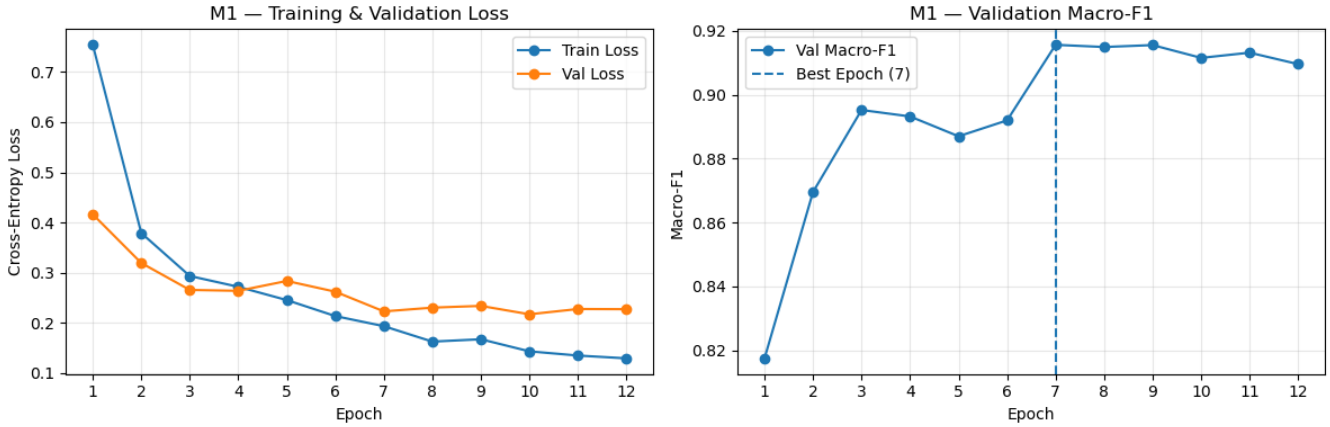


Figure 2: M1 learning curves. Validation macro-F1 was highest at epoch 7, not at the last epoch.

M1's training loss continued to decrease after epoch 7, while validation macro-F1 did not show a consistent improvement. This is why M1 was evaluated using its saved epoch 7 checkpoint rather than its epoch 12 weights.

M2: Training and Validation Behavior

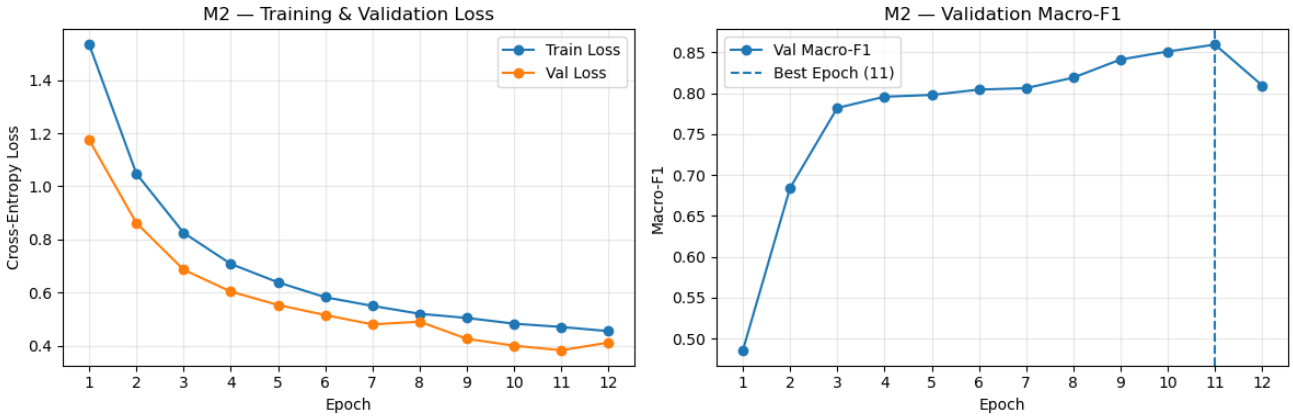


Figure 3: M2 learning curves. The highest validation macro-F1 occurred at epoch 11.

M2's training loss declined steadily, while validation macro-F1 improved more gradually and fluctuated. Its best checkpoint was saved at epoch 11. The selected epoch differed across models even though all were trained for 12 epochs. This illustrates why saved validation-selected checkpoints were used for final testing.

8. M2 Confusion Matrix and Error Analysis

Figure 4 shows the confusion matrix for M2 on the official test split.

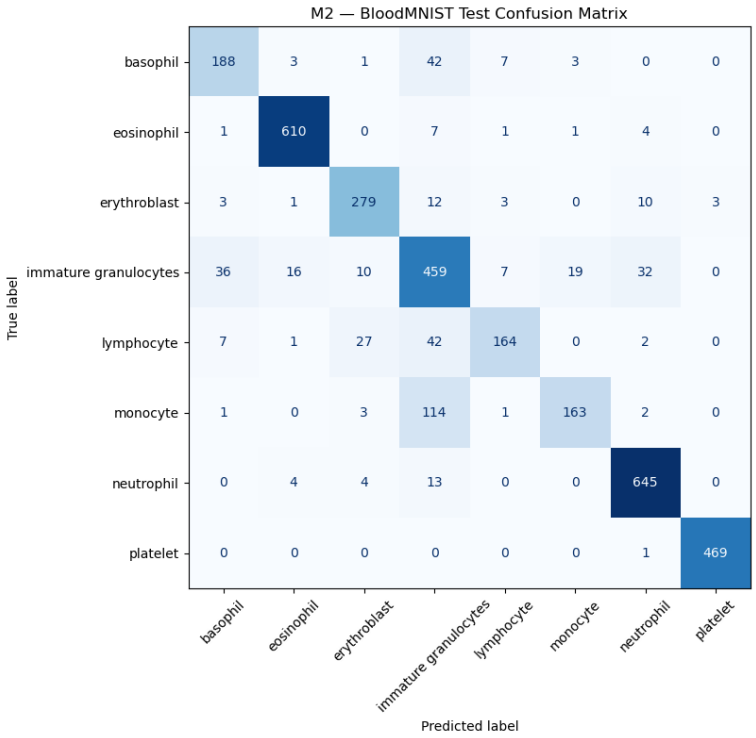


Figure 4: M2 confusion matrix on the 3,421-image official test split. Rows are true classes and columns are predicted classes.

M2 correctly classified 2,977 of 3,421 test images (accuracy 0.8702), but the confusion matrix shows substantial differences in recall across classes.

Monocytes had the lowest class recall: 163 correct predictions out of 284 monocyte images (57.39%). The most frequent single confusion was monocyte → immature granulocyte, which occurred 114 times. Immature granulocytes were also confused with basophils and neutrophils. In contrast, platelets (469/470) and eosinophils (610/624) had much stronger recall.

9. Discussion of RQ1 and RQ2

RQ1 — MLP vs. CNN. Both tested CNNs outperformed the MLP baseline on the held-out test set. Relative to M0, M1 increased test macro-F1 from 0.8150 to 0.9051, a difference of 0.0901 (9.01 percentage points). M2 increased test macro-F1 from 0.8150 to 0.8452, a difference of 0.0302 (3.02 points). These results suggest that the tested CNN designs were better matched to this image-classification task than the MLP baseline.

This comparison does not isolate the effect of convolution alone: M0 and the CNNs also differ in Batch Normalization and parameter counts.

RQ2 — Flatten vs. GAP. M1 and M2 share the same CNN feature-extractor design but use different classifier heads. M2 reduced the total parameter count from 44,680 to 20,104, which is 24,576 fewer parameters or 55.0% fewer than M1. Under the tested protocol, this reduction came with lower test macro-F1: 0.8452 for M2 compared with 0.9051 for M1, a difference of 0.0599 (5.99 points).

Thus, the smaller GAP-based CNN performed less well under this protocol. The result does not establish that Flatten is always preferable to GAP.

10. Limitations and Conclusion

Limitations. Each model was trained only once with seed 42, so this report does not measure seed-to-seed variability or statistical significance. The models differ in more than one architectural component, so the performance gap between M0 and the CNNs should not be interpreted as the isolated effect of convolution. BloodMNIST images are small (28×28), and the fixed learning rate and 12-epoch training budget may not be optimal for every architecture. Finally, benchmark results on BloodMNIST do not establish clinical reliability.

Conclusion. Under the fixed single-seed protocol, the two tested CNNs achieved higher test macro-F1 than the MLP baseline. M1 achieved the highest test macro-F1 (0.9051), while M2 used the fewest trainable parameters (20,104). The project demonstrates model comparison, validation-based checkpoint selection, and descriptive error analysis in PyTorch. Its findings are limited to the tested settings.

11. Reproducibility and Intended Use

The executable notebook linked in the repository documents preprocessing, model definitions, training, checkpoint selection, final test metrics, and the M2 confusion matrix. To rerun the experiment, create a project directory with `notebooks/` and `data/`, place the downloaded notebook in `notebooks/`, and execute its cells in order. The notebook saves outputs to `results/`. Use seed 42 and the fixed settings above. Differences in PyTorch/package versions or hardware backends may change the numerical results.

This is an educational experiment, not a clinical diagnostic evaluation. Additional random seeds or datasets would be useful future extensions; they were not part of the present work.

References

[1] Jiancheng Yang, Rui Shi, Donglai Wei, et al. MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10:41, 2023. doi: 10.1038/s41597-022-01721-8.

[2] Andrea Acevedo, Santiago Merino, Santiago Alférez, et al. A dataset of microscopic peripheral blood cell images for development of automatic recognition systems. *Data in Brief*, 30:105474, 2020. doi: 10.1016/j.dib.2020.105474.

Source code and results: https://github.com/uyentran9/bloodmnist-mlp-cnn. This report covers M0–M2 with seed 42. Repository artifacts outside this scope are not discussed here.

A. Model Architectures

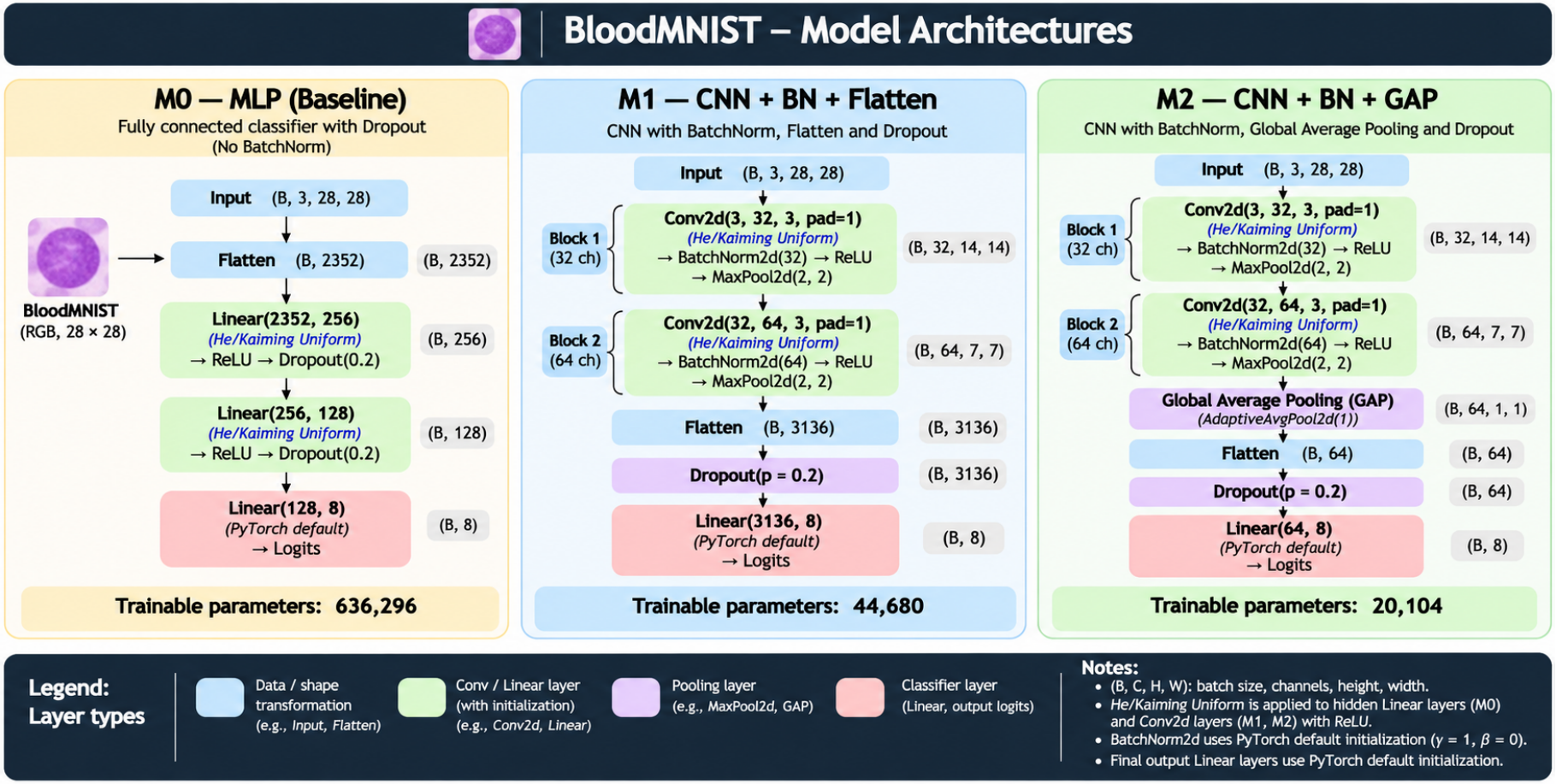


Figure 5: Model architecture overview. M0 is an MLP baseline; M1 uses a CNN with a Flatten head; M2 replaces Flatten with GAP. The diagram shows intermediate tensor shapes, initialization notes, and trainable parameter counts.

Reading the diagram. M1 flattens $64 \times 7 \times 7$ features before its Linear classifier; M2 applies GAP to obtain 64 features. The models use the same CNN feature-extractor design but are initialized and trained independently.